

tech__talks

KUBERNETES

The Complete Textbook

From Fundamentals to Production-Grade Mastery

Covers: Workloads • Networking • Security • Storage • Scheduling
Scaling • Architecture • Helm • Kustomize • Deployment Strategies

tech__talks

Table of Contents

Table of Contents	2
Chapter 1: Pods — The Atomic Unit of Kubernetes	6
1.1 What Is a Pod?.....	6
Pod Anatomy	6
Pod Lifecycle Phases	6
1.2 Single-Container vs Multi-Container Pods	7
Multi-Container Patterns	7
Sidecar Pattern	7
Ambassador Pattern	7
Adapter Pattern	7
1.3 Pod Networking.....	7
1.4 Resource Requests and Limits.....	7
1.5 Init Containers	8
1.6 kubectl Commands for Pods.....	8
1.7 Static Pods	8
Chapter 2: Replication Controller & ReplicaSet.....	9
2.1 Replication Controller (RC) — The Legacy Approach.....	9
RC Key Fields.....	9
2.2 ReplicaSet (RS) — The Modern Replacement	9
ReplicaSet Selectors	9
ReplicaSet YAML Example	10
2.3 Scaling ReplicaSets	10
2.4 ReplicaSet vs ReplicationController — Summary	10
Chapter 3: Deployments — Managing Application Lifecycles.....	11
3.1 How Deployments Work	11
3.2 Deployment YAML	11
3.3 Deployment Commands	12
3.4 Deployment Strategies	12
Rolling Update (Default).....	12
Recreate	12
Chapter 4: DaemonSet — One Pod Per Node	13
4.1 Common DaemonSet Use Cases.....	13
4.2 DaemonSet YAML.....	13
4.3 DaemonSet on Specific Nodes.....	14
4.4 DaemonSet Update Strategies	14
Chapter 5: Jobs and CronJobs — Batch Processing.....	15
5.1 Jobs	15

Job Completion Patterns.....	15
5.2 CronJobs.....	15
Chapter 6: StatefulSet — Managing Stateful Applications.....	17
6.1 StatefulSet Guarantees	17
6.2 StatefulSet YAML	17
6.3 Headless Service	18
Chapter 7: Services — Stable Network Access to Pods.....	19
7.1 Service Types	19
ClusterIP (Default)	19
NodePort	19
LoadBalancer	19
ExternalName.....	19
7.2 Service YAML Examples	20
7.3 Endpoints and EndpointSlices	20
Chapter 8: Ingress & Gateway API	21
8.1 Ingress	21
How Ingress Works.....	21
8.2 Gateway API	22
Gateway API Resources.....	22
Chapter 9: Pod Networking & CNI Plugins	23
9.1 Kubernetes Networking Requirements	23
9.2 CNI Plugin Overview	23
9.3 Calico	23
9.4 Cilium	23
9.5 DNS in Kubernetes.....	24
Chapter 10: Security — RBAC, Service Accounts & Security Context	25
10.1 RBAC — Role-Based Access Control	25
10.2 Service Accounts.....	26
10.3 Security Context.....	26
10.4 Network Policies.....	26
10.5 Secrets.....	27
Chapter 11: Cluster Architecture & Control Plane Components	28
11.1 API Server (kube-apiserver)	28
Request Lifecycle	28
11.2 etcd — The Cluster Database	28
etcd Backup & Restore	28
11.3 Scheduler (kube-scheduler).....	29
Scheduling Process	29
11.4 Controller Manager (kube-controller-manager)	29

11.5 kubelet	29
11.6 kube-proxy	29
11.7 Certificates in Kubernetes	30
Chapter 12: Storage — Persistent Volumes, Claims & Storage Classes.....	31
12.1 Persistent Volume (PV)	31
PV Access Modes.....	31
PV Reclaim Policies.....	31
12.2 Persistent Volume Claim (PVC).....	31
12.3 Storage Class (SC)	32
12.4 Volume Types	32
12.5 emptyDir.....	32
12.6 hostPath.....	32
Chapter 13: Advanced Scheduling — Taints, Affinity & Anti-Affinity	34
13.1 Taints & Tolerations.....	34
Taint Effects.....	34
13.2 Node Selector.....	34
13.3 Node Affinity	35
13.4 Pod Affinity & Anti-Affinity	35
Use Cases	35
Chapter 14: Autoscaling — HPA, VPA & Metrics Server.....	36
14.1 Metrics Server	36
14.2 Horizontal Pod Autoscaler (HPA).....	36
14.3 Vertical Pod Autoscaler (VPA).....	37
Chapter 15: API Groups, CRDs, Helm & Kustomize	38
15.1 Kubernetes API Groups.....	38
15.2 Custom Resource Definitions (CRD)	38
15.3 Helm — The Kubernetes Package Manager	39
Core Helm Concepts	39
15.4 Kustomize — Configuration Customization.....	39
Kustomize Structure	39
Chapter 16: Deployment Strategies	41
16.1 Rolling Update.....	41
16.2 Recreate.....	41
16.3 Blue-Green Deployment	41
16.4 Canary Deployment.....	41
Chapter 17: Probes — Liveness, Readiness & Startup	43
17.1 Liveness Probe.....	43
17.2 Readiness Probe	43
17.3 Startup Probe	43

17.4 Probe Configuration	43
Probe Mechanisms	44
Appendix A: Diagram Instructions	45
A.1 General Diagram Style Guide	45
A.2 Kubernetes Architecture Overview (Chapter 11)	45
A.3 Service Traffic Flow (Chapter 7)	45
Appendix B: Complete kubectl Command Reference	46
B.1 Context & Cluster Management	46
B.2 Namespace Commands	46
B.3 Workload Commands	46
B.4 Debugging Commands	46
Appendix C: Glossary of Kubernetes Terms	47

Chapter 1: Pods — The Atomic Unit of Kubernetes

A Pod is the smallest deployable unit in Kubernetes. Just as an atom is the smallest unit of matter, a Pod is the smallest unit Kubernetes can schedule, manage, and orchestrate. Every application you run in Kubernetes lives inside a Pod.

□ **Key Analogy**

Think of a Pod like a shipping container on a cargo ship. The container (Pod) holds one or more items (containers/processes) that always travel together, share the same space, and are loaded/unloaded as a single unit.

1.1 What Is a Pod?

A Pod wraps one or more containers that share the same network namespace (IP address and port space), storage volumes, and lifecycle. Containers within the same Pod communicate via localhost — they are co-located processes.

Pod Anatomy

Every Pod specification includes:

- metadata — name, namespace, labels, annotations
- spec — the desired state: which containers to run, what images to use, resource limits, volumes
- status — the current state: phase (Pending, Running, Succeeded, Failed, Unknown), conditions, container statuses

□ **DIAGRAM: Pod Architecture**

Box labeled 'Pod' containing: two container boxes (Container A, Container B) sharing a 'Shared Network (localhost)' layer, a 'Shared Volumes' disk icon at the bottom, and the Pod's IP address shown on the outside. Arrows from Container A and B both pointing to 'localhost' internally.

[Insert diagram here — instructions in Appendix C]

Pod Lifecycle Phases

Phase	Meaning	Example Scenario
Pending	Pod accepted but containers not yet started	Waiting for image pull or node resources
Running	At least one container is running	App serving traffic normally
Succeeded	All containers exited with code 0	Batch job completed
Failed	At least one container exited non-zero	App crashed, no restart policy
Unknown	Pod state cannot be determined	Node communication lost

1.2 Single-Container vs Multi-Container Pods

The most common pattern is one container per Pod, keeping concerns separated. However, Kubernetes supports multi-container Pods for tightly coupled processes that must share resources.

Multi-Container Patterns

Sidecar Pattern

A helper container enhances the main container. Example: a log-shipping sidecar reads logs written by the main app and forwards them to a centralized logging system. The main app writes logs; the sidecar ships them — they work together but are independently deployable.

Ambassador Pattern

A proxy container handles outbound network calls on behalf of the main container. Example: the main app thinks it is talking to localhost:6379 (Redis), but the ambassador container proxies the request to the actual Redis cluster with connection pooling and retries.

Adapter Pattern

A transformation container normalizes the output of the main container. Example: a legacy app outputs metrics in a proprietary format; the adapter converts them to Prometheus format for scraping.

□ DIAGRAM: Multi-Container Pod Patterns

Three side-by-side boxes each showing a Pod: (1) Sidecar — Main Container + Log Shipper sharing a Volume; (2) Ambassador — Main Container + Proxy Container, with arrow from proxy to external Redis; (3) Adapter — Legacy App + Metrics Converter, with arrow from converter to Prometheus.

[Insert diagram here — instructions in Appendix C]

1.3 Pod Networking

Every Pod gets its own unique IP address within the cluster. Pods can communicate with each other directly using these IPs — there is no NAT within a cluster. This flat network model is enforced by the CNI plugin.

- Containers within the same Pod share the Pod's IP — they talk via localhost
- Pods on the same node communicate directly via the Pod network
- Pods on different nodes communicate through the cluster network overlay

1.4 Resource Requests and Limits

Kubernetes uses requests and limits to manage CPU and memory allocation. Requests are what the scheduler uses to find a node; limits are enforced by the container runtime.

```
# Pod resource specification example
resources:
  requests:
    memory: "128Mi"    # Guaranteed minimum memory
    cpu: "250m"        # 0.25 CPU cores guaranteed
  limits:
    memory: "256Mi"    # Maximum memory allowed
```

```
cpu: "500m"           # Maximum 0.5 CPU cores
```

❑ Important

If a container exceeds its memory limit, it is OOMKilled (Out Of Memory killed). If it exceeds CPU limit, it is throttled — not killed. Always set both requests and limits for production workloads.

1.5 Init Containers

Init containers run before the main application containers in a Pod. They must complete successfully before the main containers start. Common uses: database migration, waiting for a dependency to be ready, seeding configuration files.

❑ Analogy

Init containers are like mise en place in cooking — the prep work (chopping vegetables, measuring spices) that must be done before the main cooking begins. The main chef (app container) only starts once the prep is complete.

1.6 kubectl Commands for Pods

```
kubectl run nginx --image=nginx           # Create a Pod
kubectl get pods                          # List Pods
kubectl get pods -o wide                  # List with node info
kubectl describe pod <name>               # Full details & events
kubectl logs <pod-name>                   # View container logs
kubectl logs <pod-name> -c <container>    # Specific container logs
kubectl exec -it <pod-name> -- /bin/bash  # Shell into Pod
kubectl delete pod <name>                 # Delete Pod
kubectl get pod <name> -o yaml             # Export Pod YAML
```

1.7 Static Pods

Static Pods are managed directly by the kubelet on a node, not by the API server. They are defined in files on the node's filesystem (typically `/etc/kubernetes/manifests/`). The kubelet watches this directory and creates/restarts static Pods automatically. Control plane components (API server, etcd, scheduler, controller manager) run as static Pods.

❑ DIAGRAM: Static Pod Flow

Left side: a Node box containing kubelet watching `/etc/kubernetes/manifests/` folder. Arrow from folder to kubelet. Arrow from kubelet to 'Static Pod' box. Right side: API Server showing a mirror Pod (read-only) for visibility. Note: 'No Deployment/ReplicaSet controls static Pods'.

[Insert diagram here — instructions in Appendix C]

Chapter 2: Replication Controller & ReplicaSet

Running a single Pod is rarely sufficient for production workloads. If a Pod crashes, it is gone forever. Kubernetes solved this with controllers that ensure a specified number of Pod replicas are running at any given time.

2.1 Replication Controller (RC) — The Legacy Approach

The Replication Controller was the first mechanism for maintaining Pod replicas. It ensures that exactly N copies of a Pod are running. If a Pod fails, the RC creates a new one. If there are too many Pods, it deletes the excess.

❏ Deprecation Note

Replication Controller is considered legacy as of Kubernetes 1.2+. It has been superseded by ReplicaSet (which supports set-based label selectors). New workloads should use ReplicaSet or Deployment.

RC Key Fields

- replicas — desired number of Pod copies
- selector — which Pods this RC manages (equality-based only: key=value)
- template — the Pod template used to create new Pods

2.2 ReplicaSet (RS) — The Modern Replacement

ReplicaSet is the upgraded version of Replication Controller. The critical difference is support for set-based selectors (In, NotIn, Exists), enabling more powerful Pod selection. ReplicaSets are almost never used directly — they are typically managed by a Deployment.

❏ DIAGRAM: ReplicaSet Architecture

Central box 'ReplicaSet Controller' with three Pod boxes below it labeled Pod-1, Pod-2, Pod-3. Arrows from controller to each Pod. A dashed box shows 'Pod Template'. If Pod-2 is deleted (shown with X), a new Pod-4 appears with an arrow from the controller. Label: 'Desired: 3, Actual: 3'.

[Insert diagram here — instructions in Appendix C]

ReplicaSet Selectors

Replication Controller (Equality)	ReplicaSet (Set-Based)
app=nginx	app in (nginx, apache)
tier=frontend	tier notin (backend)
Cannot use complex logic	Supports Exists, DoesNotExist
Deprecated approach	Recommended approach

ReplicaSet YAML Example

```
apiVersion: apps/v1
kind: ReplicaSet
metadata:
  name: nginx-rs
spec:
  replicas: 3
  selector:
    matchLabels:
      app: nginx
  template:
    metadata:
      labels:
        app: nginx
    spec:
      containers:
        - name: nginx
          image: nginx:1.25
          ports:
            - containerPort: 80
```

2.3 Scaling ReplicaSets

```
kubectl scale rs nginx-rs --replicas=5      # Scale up
kubectl scale rs nginx-rs --replicas=1      # Scale down
kubectl get rs                               # View ReplicaSets
kubectl describe rs nginx-rs                # Detailed info
kubectl delete rs nginx-rs                  # Delete RS (deletes Pods too)
kubectl delete rs nginx-rs --cascade=orphan  # Delete RS, keep Pods
```

2.4 ReplicaSet vs ReplicationController — Summary

Feature	ReplicationController	ReplicaSet
API Version	v1	apps/v1
Selector Type	Equality-based	Set-based + Equality
Used Directly?	Sometimes (legacy)	Rarely (via Deployment)
Status	Deprecated	Current

Chapter 3: Deployments — Managing Application Lifecycles

A Deployment is the most commonly used Kubernetes object for stateless applications. It provides declarative updates, rollback capability, and automated rollout strategies on top of ReplicaSets.

□ Analogy

A Deployment is like a project manager who oversees a team (ReplicaSet) of workers (Pods). The manager handles hiring (scaling), firing (termination), and smoothly onboarding new team members when the project requirements change (rolling updates).

3.1 How Deployments Work

When you create or update a Deployment, it creates a ReplicaSet. The Deployment controller monitors the desired state and reconciles any deviations. When you update the Pod template (e.g., change the image), the Deployment creates a new ReplicaSet and gradually shifts traffic from old to new.

□ DIAGRAM: Deployment Hierarchy

Top box: 'Deployment'. Arrow down to 'ReplicaSet v2 (current, 3 replicas)' and a faded 'ReplicaSet v1 (old, 0 replicas)'. Arrows from ReplicaSet v2 to three Pod boxes. Note: 'Deployment retains old ReplicaSets for rollback'. History depth shown as revision 1, 2, 3.

[Insert diagram here — instructions in Appendix C]

3.2 Deployment YAML

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: my-app
  namespace: production
spec:
  replicas: 3
  selector:
    matchLabels:
      app: my-app
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxSurge: 1      # Max extra Pods during update
      maxUnavailable: 0 # Min available during update
  template:
    metadata:
      labels:
        app: my-app
    spec:
      containers:
        - name: app
          image: myapp:v2
          ports:
            - containerPort: 8080
```

3.3 Deployment Commands

```
kubectl create deployment nginx --image=nginx           # Create
kubectl set image deployment/nginx nginx=nginx:1.25     # Update image
kubectl rollout status deployment/nginx                 # Watch rollout
kubectl rollout history deployment/nginx                # View history
kubectl rollout undo deployment/nginx                   # Rollback
kubectl rollout undo deployment/nginx --to-revision=2   # Rollback to v2
kubectl rollout pause deployment/nginx                  # Pause rollout
kubectl rollout resume deployment/nginx                 # Resume rollout
kubectl scale deployment nginx --replicas=5            # Scale
```

3.4 Deployment Strategies

Rolling Update (Default)

Gradually replaces old Pods with new ones. At no point are all Pods unavailable. Controlled by maxSurge and maxUnavailable. Best for zero-downtime deployments.

Recreate

All existing Pods are killed before new ones are created. Causes downtime but ensures no two versions run simultaneously. Use for applications that cannot run mixed versions.

□ Deep Dive

maxSurge=1 means at most 1 extra Pod can exist during the update (3 desired + 1 surge = max 4 Pods). maxUnavailable=0 means all 3 desired Pods must always be available. Together these ensure zero-downtime with controlled resource usage.

Chapter 4: DaemonSet — One Pod Per Node

A DaemonSet ensures that a copy of a Pod runs on every (or selected) node in the cluster. As nodes are added, Pods are automatically added. As nodes are removed, Pods are garbage collected.

□ Analogy

A DaemonSet is like a security guard assigned to every floor of a building. Each floor (node) always has exactly one guard (Pod), no matter how many floors the building has. When a new floor is added, a guard is automatically assigned.

4.1 Common DaemonSet Use Cases

- Log collectors (Fluentd, Filebeat) — must run on every node to collect logs
- Node monitoring agents (Prometheus Node Exporter, Datadog agent)
- Network plugins (CNI agents like Calico, Cilium must run on every node)
- Storage agents (Ceph, GlusterFS daemons)
- Security scanning agents

□ DIAGRAM: DaemonSet Distribution

A cluster with 4 nodes (Node 1, 2, 3, 4). Each node has a DaemonSet Pod (shown as DS-Pod). When a new 'Node 5' is added (shown with dashed border), an arrow from 'DaemonSet Controller' automatically creates 'DS-Pod' on it. Arrow shows automatic placement.

[Insert diagram here — instructions in Appendix C]

4.2 DaemonSet YAML

```
apiVersion: apps/v1
kind: DaemonSet
metadata:
  name: node-exporter
  namespace: monitoring
spec:
  selector:
    matchLabels:
      app: node-exporter
  template:
    metadata:
      labels:
        app: node-exporter
    spec:
      tolerations:
        - key: node-role.kubernetes.io/control-plane
          operator: Exists
          effect: NoSchedule
      containers:
        - name: node-exporter
          image: prom/node-exporter:latest
          ports:
```

```
- containerPort: 9100
```

4.3 DaemonSet on Specific Nodes

By default, DaemonSets run on all nodes. Use `nodeSelector` or `nodeAffinity` to restrict to specific nodes. Add tolerations to run on tainted nodes (like control-plane nodes).

4.4 DaemonSet Update Strategies

OnDelete	RollingUpdate
Manual deletion triggers update	Automatic controlled rollout
Full control over timing	<code>maxUnavailable</code> controls pace
No automatic rollout	Default strategy

Chapter 5: Jobs and CronJobs — Batch Processing

While Deployments and DaemonSets manage long-running services, Jobs and CronJobs handle batch workloads — tasks that run to completion and then stop.

5.1 Jobs

A Job creates one or more Pods and ensures that a specified number of them successfully complete. Unlike Deployments, Jobs do not restart completed Pods — they track successful completions.

□ Analogy

A Job is like a contractor hired for a specific project. Once the project is done (Pod completes), the contract ends. Unlike a full-time employee (Deployment), the contractor is not expected to keep working indefinitely.

Job Completion Patterns

Pattern	completions	parallelism	Use Case
Single Job	1	1	One-time task (DB migration)
Fixed Completion	5	1	Process 5 items sequentially
Work Queue	5	3	Process 5 items with 3 workers
Indexed Job	5	5	Each Pod handles one item by index

```
apiVersion: batch/v1
kind: Job
metadata:
  name: data-migration
spec:
  completions: 1
  backoffLimit: 4      # Retry up to 4 times on failure
  template:
    spec:
      restartPolicy: Never # Required for Jobs (Never or OnFailure)
      containers:
      - name: migrator
        image: my-migrator:latest
        command: ['python', 'migrate.py']
```

5.2 CronJobs

CronJobs create Jobs on a schedule defined using standard Unix cron syntax. They are ideal for periodic tasks like nightly reports, database backups, or cache cleanup.

```
apiVersion: batch/v1
```

```
kind: CronJob
metadata:
  name: nightly-backup
spec:
  schedule: "0 2 * * *"      # 2:00 AM every day
  concurrencyPolicy: Forbid   # Don't run if previous is still running
  successfulJobsHistoryLimit: 3
  failedJobsHistoryLimit: 1
  jobTemplate:
    spec:
      template:
        spec:
          restartPolicy: OnFailure
          containers:
            - name: backup
              image: backup-tool:latest
```

□ Cron Syntax Reference

Format: minute hour day-of-month month day-of-week '0 2 * * *' = 2:00 AM daily | '*/5 * * * *' = every 5 minutes | '0 0 * * 0' = midnight every Sunday | '@hourly' = shorthand for 0 * * * *

Chapter 6: StatefulSet — Managing Stateful Applications

StatefulSets are designed for applications that require stable network identities, persistent storage, and ordered deployment/scaling. They are the go-to controller for databases, message queues, and other stateful workloads.

□ Analogy

A Deployment is like an anonymous worker pool — any worker is interchangeable. A StatefulSet is like a named team with fixed roles: Alice is always the primary database, Bob is always replica-1, Charlie is always replica-2. Their identities and responsibilities are permanent.

6.1 StatefulSet Guarantees

- Stable, unique network identities — each Pod gets a predictable hostname: <name>-0, <name>-1, <name>-2
- Stable persistent storage — each Pod gets its own PVC that survives Pod restarts
- Ordered deployment — Pods are created sequentially (0, then 1, then 2)
- Ordered scaling — scale down removes highest-ordinal Pod first
- Ordered rolling updates — updates proceed in reverse ordinal order

□ DIAGRAM: StatefulSet vs Deployment

Left side (Deployment): Three identical Pod boxes with random names (pod-abc12, pod-xyz34, pod-qwerty). One shared or no persistent storage. Right side (StatefulSet): Three Pods named mysql-0, mysql-1, mysql-2. Each Pod has its own dedicated PVC (pvc-mysql-0, pvc-mysql-1, pvc-mysql-2). mysql-0 labeled 'Primary', others 'Replica'.

[Insert diagram here — instructions in Appendix C]

6.2 StatefulSet YAML

```
apiVersion: apps/v1
kind: StatefulSet
metadata:
  name: mysql
spec:
  serviceName: "mysql"      # Headless Service name
  replicas: 3
  selector:
    matchLabels:
      app: mysql
  template:
    metadata:
      labels:
        app: mysql
    spec:
      containers:
        - name: mysql
          image: mysql:8.0
          volumeMounts:
```

```
- name: data
  mountPath: /var/lib/mysql
volumeClaimTemplates:
- metadata:
  name: data
  spec:
    accessModes: ["ReadWriteOnce"]
    resources:
      requests:
        storage: 10Gi
```

6.3 Headless Service

StatefulSets require a Headless Service (clusterIP: None) to control the DNS domain of the Pods. Each Pod gets a stable DNS entry: <pod-name>.<service-name>.<namespace>.svc.cluster.local. This enables direct Pod-to-Pod communication by stable name.

```
apiVersion: v1
kind: Service
metadata:
  name: mysql
spec:
  clusterIP: None          # Headless — no load balancing
  selector:
    app: mysql
  ports:
    - port: 3306
```

Chapter 7: Services — Stable Network Access to Pods

Pods are ephemeral — they get new IP addresses every time they restart. Services provide a stable endpoint (virtual IP + DNS name) that routes traffic to a dynamic set of Pods. Services are the backbone of Kubernetes networking.

□ **Analogy**

A Service is like a customer service phone number. No matter which agent (Pod) answers your call, you always dial the same number (Service IP/DNS). Agents come and go, but the number never changes.

7.1 Service Types

ClusterIP (Default)

Exposes the Service on an internal cluster-only IP. Only accessible from within the cluster. Used for internal microservice communication. Each Service gets a stable DNS name: <service-name>.<namespace>.svc.cluster.local.

NodePort

Exposes the Service on each node's IP at a static port (30000-32767). External traffic can reach the Service via <NodeIP>:<NodePort>. Automatically creates a ClusterIP Service as well. Suitable for development/testing but not recommended for production external access.

LoadBalancer

Exposes the Service externally using a cloud provider's load balancer (AWS ELB, GCP LB, Azure LB). Automatically provisions and configures the external load balancer. Includes NodePort and ClusterIP functionality. The standard way to expose services externally in cloud environments.

ExternalName

Maps the Service to an external DNS name (e.g., database.example.com). Returns a CNAME record instead of a proxy. Used to give Kubernetes Pods stable internal access to external services without knowing the external address.

□ **DIAGRAM: Service Types Comparison**

Four boxes stacked with connectivity arrows: (1) ExternalName → external DNS. (2) ClusterIP → internal Pods only (with 'Cluster boundary' wall). (3) NodePort → Node IP + port (30000-32767) → ClusterIP → Pods. (4) LoadBalancer → Cloud LB → Node IPs → NodePort → ClusterIP → Pods. Internet traffic enters only through LoadBalancer.

[Insert diagram here — instructions in Appendix C]

Type	Scope	Use Case	External Access
ClusterIP	Internal cluster only	Microservice-to-microservice	No
NodePort	Node IP + port	Dev/test external access	Yes (NodeIP:Port)

LoadBalancer	Cloud provider LB	Production external access	Yes (LB IP)
ExternalName	External DNS	Map to external service	Outbound only

7.2 Service YAML Examples

```
# ClusterIP Service
apiVersion: v1
kind: Service
metadata:
  name: my-service
spec:
  type: ClusterIP          # Default
  selector:
    app: my-app            # Routes to Pods with this label
  ports:
    - port: 80             # Service port
      targetPort: 8080     # Container port
---
# NodePort Service
spec:
  type: NodePort
  ports:
    - port: 80
      targetPort: 8080
      nodePort: 30080      # Optional: specify or auto-assigned
---
# LoadBalancer Service
spec:
  type: LoadBalancer
  ports:
    - port: 443
      targetPort: 8443
```

7.3 Endpoints and EndpointSlices

Behind every Service is an Endpoints (or EndpointSlice) object that holds the current list of Pod IPs and ports that match the Service selector. The kube-proxy watches EndpointSlices to program iptables/IPVS rules on each node. When a Pod is added or removed, the EndpointSlice updates automatically.

Chapter 8: Ingress & Gateway API

Services of type LoadBalancer are powerful but create one load balancer per service — expensive in cloud environments. Ingress provides HTTP/HTTPS routing from a single load balancer to multiple Services based on host names and URL paths.

8.1 Ingress

□ Analogy

Ingress is like a hotel receptionist who directs guests to the right room. A single receptionist (load balancer) handles all guests (HTTP requests) and routes them to the right floor/room based on their name (hostname) and purpose (URL path).

How Ingress Works

An Ingress object defines routing rules. An Ingress Controller (nginx, Traefik, HAProxy, AWS ALB) watches Ingress objects and configures the underlying load balancer accordingly. Without an Ingress Controller, Ingress objects have no effect.

□ DIAGRAM: Ingress Architecture

Internet → Cloud Load Balancer → Ingress Controller (nginx Pod). From Ingress Controller: path '/api' routes to 'api-service', path '/web' routes to 'web-service', host 'admin.example.com' routes to 'admin-service'. All Services forward to their respective Pods. TLS termination shown at Ingress Controller.

[Insert diagram here — instructions in Appendix C]

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: main-ingress
  annotations:
    nginx.ingress.kubernetes.io/rewrite-target: /
spec:
  ingressClassName: nginx
  tls:
  - hosts:
    - example.com
    secretName: tls-secret
  rules:
  - host: example.com
    http:
      paths:
      - path: /api
        pathType: Prefix
        backend:
          service:
            name: api-service
            port:
              number: 80
      - path: /
        pathType: Prefix
```

```
backend:
  service:
    name: web-service
    port:
      number: 80
```

8.2 Gateway API

Gateway API is the next-generation successor to Ingress. It provides a more expressive, role-oriented, and extensible API for routing. It separates concerns between infrastructure providers (GatewayClass), cluster operators (Gateway), and application developers (HTTPRoute, TCPRoute, etc.).

Gateway API Resources

Resource	Owned By	Purpose
GatewayClass	Infrastructure Provider	Defines the type of gateway (nginx, istio, etc.)
Gateway	Cluster Operator	Instantiates a gateway with listeners (ports, protocols)
HTTPRoute	Application Developer	Defines HTTP routing rules
TCPRoute	Application Developer	Defines TCP routing rules

Chapter 9: Pod Networking & CNI Plugins

Kubernetes networking is built on a flat, all-to-all Pod network model. Every Pod can communicate with every other Pod without NAT. This is enforced by CNI (Container Network Interface) plugins installed on each node.

9.1 Kubernetes Networking Requirements

- Every Pod gets a unique cluster-wide IP address
- Pods can communicate with all other Pods without NAT
- Agents on a node can communicate with all Pods on that node
- Pods on the same node can talk to each other via a virtual bridge

9.2 CNI Plugin Overview

CNI plugins implement the network requirements above. They create virtual network interfaces in Pods, assign IP addresses, and set up routing rules. Popular CNI plugins have different strengths.

□ **DIAGRAM: CNI Network Model**

Two nodes side by side. Each node has: a 'cbr0' virtual bridge, three Pod boxes connected to the bridge, and a 'veth' pair for each Pod. Between nodes: an 'Overlay Network' or 'BGP' link. IP ranges shown: Node1=10.244.1.0/24, Node2=10.244.2.0/24. Pods can ping each other across nodes.

[Insert diagram here — instructions in Appendix C]

CNI Plugin	Mechanism	Network Policy	Best For
Calico	BGP routing / overlay	Yes (native)	Large clusters, network policy needs
Flannel	VXLAN overlay	No (use Canal)	Simple setups, learning
Cilium	eBPF-based	Yes (L7 aware)	High performance, observability
Weave	Mesh overlay	Yes	Ease of use, multicast support

9.3 Calico

Calico uses BGP (Border Gateway Protocol) to route traffic between nodes without encapsulation overhead. It is the most widely deployed CNI for production clusters and offers rich NetworkPolicy support with its own CRDs (GlobalNetworkPolicy, HostEndpoint).

9.4 Cilium

Cilium leverages eBPF (extended Berkeley Packet Filter) — a technology that runs sandboxed programs in the Linux kernel without changing kernel source code. This enables extremely efficient packet processing, deep observability, and Layer 7 (application-layer) network policies.

□ eBPF Insight

Traditional network processing happens in user space (slow, copies). eBPF moves processing into the kernel directly, bypassing iptables entirely. Cilium with eBPF can handle millions of connections per second with minimal CPU overhead.

9.5 DNS in Kubernetes

CoreDNS runs as a Deployment in the kube-system namespace and provides DNS resolution for all cluster resources. Service DNS format: <service>.<namespace>.svc.cluster.local. Pod DNS format (when enabled): <pod-ip>.<namespace>.pod.cluster.local.

Chapter 10: Security — RBAC, Service Accounts & Security Context

Kubernetes security follows a principle of least privilege. Every component, user, and process should have only the minimum permissions required to do its job. RBAC (Role-Based Access Control) is the primary mechanism for controlling access.

10.1 RBAC — Role-Based Access Control

RBAC grants permissions to users, groups, and service accounts based on their role. It uses four main resources: Role, ClusterRole, RoleBinding, ClusterRoleBinding.

□ **DIAGRAM: RBAC Model**

Center: 'Subject' box (User / Group / ServiceAccount). Arrow from Subject to 'RoleBinding' (namespace-scoped) which points to 'Role' listing verbs (get, list, create) on resources (pods, deployments). Separately: 'ClusterRoleBinding' connecting to 'ClusterRole' with cluster-wide permissions. Note: Role = namespace, ClusterRole = cluster-wide.

[Insert diagram here — instructions in Appendix C]

Resource	Scope	Purpose
Role	Namespace	Grants permissions within one namespace
ClusterRole	Cluster-wide	Grants permissions across all namespaces or non-namespaced resources
RoleBinding	Namespace	Binds a Role or ClusterRole to subjects within a namespace
ClusterRoleBinding	Cluster-wide	Binds a ClusterRole to subjects cluster-wide

```
# Role - allows read access to pods in 'default' namespace
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  namespace: default
  name: pod-reader
rules:
- apiGroups: [""]
  resources: ["pods"]
  verbs: ["get", "watch", "list"]
---
# RoleBinding - assigns the Role to a user
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  name: read-pods
  namespace: default
subjects:
- kind: User
```

```
name: jane
apiGroup: rbac.authorization.k8s.io
roleRef:
  kind: Role
  name: pod-reader
apiGroup: rbac.authorization.k8s.io
```

10.2 Service Accounts

Service Accounts provide an identity for processes running in Pods. Every Pod is associated with a ServiceAccount (default: 'default'). Kubernetes automatically mounts a JWT token into Pods that allows them to authenticate with the API server.

- Create a dedicated ServiceAccount per application component
- Bind only the minimum required permissions via RBAC
- Use automountServiceAccountToken: false when API access is not needed

10.3 Security Context

A SecurityContext defines privilege and access control settings for a Pod or individual container. It controls how the container process interacts with the underlying OS.

```
spec:
  securityContext:
    # Pod-level
    runAsUser: 1000          # Run as non-root UID
    runAsGroup: 3000        # Run as GID 3000
    fsGroup: 2000           # Volume files owned by GID 2000
  containers:
    - name: app
      securityContext:
        # Container-level (overrides pod-level)
        allowPrivilegeEscalation: false
        readOnlyRootFilesystem: true
        capabilities:
          drop: ["ALL"]
          add: ["NET_BIND_SERVICE"]
```

10.4 Network Policies

By default, all Pods in a cluster can communicate with each other freely. Network Policies restrict this by defining ingress (incoming) and egress (outgoing) rules. Only CNI plugins that support Network Policies (Calico, Cilium, Weave) enforce them — Flannel does not.

```
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: deny-all
  namespace: production
spec:
  podSelector: {}          # Applies to all Pods
  policyTypes:
    - Ingress
```

```
- Egress
# No ingress/egress rules = deny all
```

10.5 Secrets

Secrets store sensitive data such as passwords, API keys, and TLS certificates. They are base64-encoded (not encrypted by default) and mounted into Pods as environment variables or files.

❑ Security Note

Base64 encoding is NOT encryption. Anyone with read access to Secrets in a namespace can decode them. For true secret management, use encryption at rest (etcd encryption), enable audit logging, integrate with external secret managers (Vault, AWS Secrets Manager), and restrict RBAC access to Secrets strictly.

```
kubectl create secret generic db-secret \
  --from-literal=username=admin \
  --from-literal=password=supersecret

# Use in Pod as env var
env:
- name: DB_PASSWORD
  valueFrom:
    secretKeyRef:
      name: db-secret
      key: password
```

Chapter 11: Cluster Architecture & Control Plane Components

A Kubernetes cluster consists of a Control Plane (the brain) and Worker Nodes (the muscles). Understanding how these components interact is essential for troubleshooting, capacity planning, and passing the CKA exam.

□ DIAGRAM: Kubernetes Cluster Architecture

Top section 'Control Plane': API Server (center) connected to etcd (data store), Scheduler, Controller Manager, and Cloud Controller Manager. Below: 'Worker Node' box containing kubelet, kube-proxy, Container Runtime (containerd), and multiple Pod boxes. Arrows: `kubectl` → API Server, API Server ↔ etcd, API Server → Scheduler, API Server → Controller Manager, kubelet ↔ API Server, kube-proxy ↔ API Server.

[Insert diagram here — instructions in Appendix C]

11.1 API Server (kube-apiserver)

The API server is the front-end for the Kubernetes control plane. Every operation (`kubectl` commands, controller actions, webhook calls) goes through the API server. It validates and processes REST requests, updates etcd, and serves as the central communication hub.

Request Lifecycle

1. Authentication — who are you? (certificates, tokens, OIDC)
2. Authorization — are you allowed? (RBAC, ABAC)
3. Admission Control — should this be allowed? (webhooks, policies)
4. Validation — is the object schema valid?
5. Persistence — write to etcd

11.2 etcd — The Cluster Database

etcd is a distributed, consistent key-value store that holds all cluster state: Pods, Services, Secrets, ConfigMaps, RBAC rules — everything. It uses the Raft consensus algorithm to ensure consistency across etcd nodes.

□ Analogy

etcd is the cluster's single source of truth — like a company's central database. If etcd is lost without a backup, the entire cluster configuration is gone. Backing up etcd is one of the most critical operational tasks in Kubernetes.

etcd Backup & Restore

```
# Backup etcd snapshot
ETCDCTL_API=3 etcdctl snapshot save /backup/etcd-snapshot.db \
  --endpoints=https://127.0.0.1:2379 \
  --cacert=/etc/etcd/ca.crt \
  --cert=/etc/etcd/server.crt \
```

```
--key=/etc/etcd/server.key

# Verify snapshot
ETCDCTL_API=3 etcdctl snapshot status /backup/etcd-snapshot.db

# Restore from snapshot
ETCDCTL_API=3 etcdctl snapshot restore /backup/etcd-snapshot.db \
  --data-dir=/var/lib/etcd-restored
```

11.3 Scheduler (kube-scheduler)

The scheduler watches for newly created Pods with no assigned node and selects the best node for them to run on. It considers resource requirements, hardware/software constraints, affinity/anti-affinity rules, taints/tolerations, and more.

Scheduling Process

6. Filtering — eliminate nodes that don't meet requirements (insufficient CPU, taints, etc.)
7. Scoring — rank remaining nodes based on multiple priority functions
8. Binding — assign the Pod to the highest-scoring node

11.4 Controller Manager (kube-controller-manager)

The controller manager runs a collection of controllers, each responsible for one aspect of cluster state. Each controller watches the API server for its resource type and reconciles the actual state to match the desired state.

Controller	Responsibility
Node Controller	Monitors node health, marks unresponsive nodes, evicts Pods
Replication Controller	Ensures correct number of Pod replicas
Endpoints Controller	Updates Endpoints objects when Pods/Services change
Service Account Controller	Creates default ServiceAccounts for new namespaces
Job Controller	Tracks Job completion, creates/deletes Pods

11.5 kubelet

The kubelet is the primary node agent. It runs on every node and ensures that containers described in PodSpecs are running and healthy. It communicates with the container runtime (containerd, CRI-O) via the CRI (Container Runtime Interface).

11.6 kube-proxy

kube-proxy runs on every node and implements the Service abstraction by maintaining network rules. It watches Services and EndpointSlices and updates iptables/IPVS rules to route Service traffic to the correct Pods.

11.7 Certificates in Kubernetes

Kubernetes uses TLS certificates for all internal communication. The Certificate Authority (CA) signs certificates for each component. kubeadm-bootstrapped clusters store CA files in /etc/kubernetes/pki/.

```
# View certificate expiration
kubeadm certs check-expiration

# Renew all certificates
kubeadm certs renew all

# Certificate locations (kubeadm cluster)
# /etc/kubernetes/pki/ca.crt           - Cluster CA
# /etc/kubernetes/pki/apiserver.crt   - API Server cert
# /etc/kubernetes/pki/etcd/ca.crt     - etcd CA
```

Chapter 12: Storage — Persistent Volumes, Claims & Storage Classes

Container filesystems are ephemeral by nature — data is lost when a container restarts. Kubernetes provides a rich storage abstraction layer that decouples storage provisioning from storage consumption.

□ DIAGRAM: Kubernetes Storage Architecture

Three-layer diagram: Top — StorageClass (AWS gp3, GCE pd-ssd, etc.) with an arrow labeled 'Dynamic Provisioning'. Middle — PersistentVolume (PV): cluster-level resource representing actual storage. Bottom — PersistentVolumeClaim (PVC): namespace-level request from a Pod. Arrows: PVC ↔ PV (binding), PV → cloud disk. Pod box with volumeMount arrow to PVC.

[Insert diagram here — instructions in Appendix C]

12.1 Persistent Volume (PV)

A PersistentVolume is a piece of storage in the cluster that has been provisioned by an administrator or dynamically via a StorageClass. It is a cluster-level resource — it exists independent of any namespace. A PV has a lifecycle independent of the Pods that use it.

PV Access Modes

Access Mode	Abbreviation	Description
ReadWriteOnce	RWO	Mounted read-write by a single node
ReadOnlyMany	ROX	Mounted read-only by many nodes
ReadWriteMany	RWX	Mounted read-write by many nodes (NFS, CephFS)
ReadWriteOncePod	RWOP	Mounted read-write by exactly one Pod

PV Reclaim Policies

- Retain — keep the PV and data after PVC is deleted; admin must manually reclaim
- Delete — automatically delete the PV and underlying storage when PVC is deleted
- Recycle — (deprecated) basic scrub and make available for new claim

12.2 Persistent Volume Claim (PVC)

A PVC is a request for storage by a user. It specifies storage size, access modes, and optionally a StorageClass. Kubernetes binds the PVC to a matching PV. If no matching PV exists and a StorageClass is specified, dynamic provisioning creates one.

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: my-pvc
spec:
  accessModes:
```

```
- ReadWriteOnce
storageClassName: standard
resources:
  requests:
    storage: 5Gi
```

12.3 Storage Class (SC)

StorageClasses enable dynamic provisioning — Kubernetes automatically creates PVs on demand. Each StorageClass references a provisioner (driver) that creates storage in the underlying infrastructure.

```
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: fast-ssd
provisioner: ebs.csi.aws.com      # AWS EBS CSI driver
parameters:
  type: gp3
  encrypted: "true"
reclaimPolicy: Delete
volumeBindingMode: WaitForFirstConsumer # Wait for Pod scheduling
allowVolumeExpansion: true
```

12.4 Volume Types

Volume Type	Description	Use Case
emptyDir	Temporary storage; lives as long as the Pod	Shared scratch space between containers
hostPath	Mounts a file/dir from the node's filesystem	Node-level data, monitoring agents
Local PV	Static local disk on a specific node	High-performance local storage
NFS	Network File System share	Shared ReadWriteMany storage
ConfigMap/Secret	Inject config/secrets as files	Configuration management

12.5 emptyDir

emptyDir volumes are created when a Pod is assigned to a node and exist as long as the Pod runs. All containers in the Pod share the volume. Data is lost when the Pod is deleted. By default, emptyDir volumes are stored on the node's disk; set medium: Memory to use a RAM-backed tmpfs.

12.6 hostPath

hostPath mounts a file or directory from the host node's filesystem into a Pod. Powerful but dangerous — a compromised Pod can access or modify the node's filesystem. Use only when absolutely necessary and restrict with SecurityContext.

Chapter 13: Advanced Scheduling — Taints, Affinity & Anti-Affinity

Kubernetes provides powerful mechanisms to control Pod placement. These tools allow you to express requirements (Pod must run on SSD nodes), preferences (Pod prefers low-latency nodes), and restrictions (Pods should not run together).

13.1 Taints & Tolerations

Taints are applied to nodes to repel Pods. Tolerations are applied to Pods to allow them to be scheduled on tainted nodes. Think of taints as a 'Do Not Disturb' sign on a node room, and tolerations as the VIP key that overrides it.

□ DIAGRAM: Taints and Tolerations

Left: Node box labeled 'gpu-node' with a taint tag 'gpu=true:NoSchedule'. Center: Two Pods — Pod A has no toleration (X mark, cannot schedule). Pod B has toleration 'gpu=true:NoSchedule' (checkmark, can schedule). Right: 'NoExecute' taint effect shown evicting an existing Pod without toleration.

[Insert diagram here — instructions in Appendix C]

Taint Effects

Effect	Behavior
NoSchedule	New Pods without toleration will not be scheduled on this node
PreferNoSchedule	Kubernetes will try to avoid scheduling Pods without toleration
NoExecute	Existing Pods without toleration are evicted; new Pods won't be scheduled

```
# Add a taint to a node
kubectl taint nodes gpu-node gpu=true:NoSchedule

# Remove a taint
kubectl taint nodes gpu-node gpu=true:NoSchedule-

# Toleration in Pod spec
tolerations:
- key: "gpu"
  operator: "Equal"
  value: "true"
  effect: "NoSchedule"
```

13.2 Node Selector

The simplest form of node selection. Add a nodeSelector to a Pod spec with key-value label pairs. The Pod will only be scheduled on nodes that have all the specified labels. Equality-based only — no complex expressions.

```
nodeSelector:
  disktype: ssd
  region: us-east-1
```

13.3 Node Affinity

Node Affinity is the powerful successor to nodeSelector. It supports set-based operators (In, NotIn, Exists, Gt, Lt) and distinguishes between required rules (hard) and preferred rules (soft).

requiredDuringSchedulingIgnoredDuringExecution	preferredDuringSchedulingIgnoredDuringExecution
Hard requirement — Pod won't schedule if not met	Soft preference — Pod schedules anyway if not met
Pod stays on node even if label is later removed	Pod stays on node even if label is later removed
Used for strict placement requirements	Used for optimization hints

13.4 Pod Affinity & Anti-Affinity

While Node Affinity attracts Pods to nodes based on node labels, Pod Affinity attracts Pods to nodes that already have certain Pods running. Pod Anti-Affinity repels Pods from nodes where certain Pods are running.

Use Cases

- Pod Affinity: co-locate a web app with its cache (Redis) on the same node for low latency
- Pod Anti-Affinity: spread replicas of the same app across different nodes/zones for HA

□ **DIAGRAM: Pod Anti-Affinity for High Availability**

Three nodes (Node 1, Node 2, Node 3) across two availability zones (Zone A, Zone B). Each node has one 'web-app' Pod. Anti-affinity rule shown: 'Do not schedule web-app Pod on a node that already has a web-app Pod'. New Pod 4 cannot go to any node already running web-app, so it triggers a new node or requeues.

[Insert diagram here — instructions in Appendix C]

Chapter 14: Autoscaling — HPA, VPA & Metrics Server

Kubernetes supports automatic scaling of workloads based on observed metrics. This ensures your applications handle varying load efficiently without manual intervention.

14.1 Metrics Server

Metrics Server is a cluster-wide aggregator of resource usage data (CPU/memory). It is a prerequisite for HPA and `kubectl` top commands. It scrapes resource metrics from kubelets and exposes them via the Metrics API.

```
# Install Metrics Server
kubectl apply -f https://github.com/kubernetes-sigs/metrics-
server/releases/latest/download/components.yaml

# View resource usage
kubectl top nodes
kubectl top pods
kubectl top pods --namespace=production
```

14.2 Horizontal Pod Autoscaler (HPA)

HPA automatically scales the number of Pod replicas in a Deployment, ReplicaSet, or StatefulSet based on observed CPU utilization, memory usage, or custom metrics. It periodically queries the Metrics API and adjusts replicas.

□ Analogy

HPA is like a restaurant manager who adds more waiters during lunch rush and sends them home after the dinner crowd leaves. The number of waiters (replicas) scales with the number of customers (load).

□ DIAGRAM: HPA Control Loop

Circular flow: (1) HPA queries Metrics API every 15s → (2) Calculates desired replicas = $\text{ceil}(\text{currentReplicas} \times \text{currentMetric}/\text{targetMetric})$ → (3) If different from current, updates Deployment replicas → (4) Deployment adjusts Pod count → (5) Load changes → back to step 1. Min/Max replica bounds shown as guardrails.

[Insert diagram here — instructions in Appendix C]

```
# Create HPA imperatively
kubectl autoscale deployment my-app --min=2 --max=10 --cpu-percent=70

# HPA YAML with multiple metrics
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: my-app-hpa
spec:
  scaleTargetRef:
```

```
apiVersion: apps/v1
kind: Deployment
name: my-app
minReplicas: 2
maxReplicas: 10
metrics:
- type: Resource
  resource:
    name: cpu
    target:
      type: Utilization
      averageUtilization: 70
```

14.3 Vertical Pod Autoscaler (VPA)

VPA automatically adjusts the CPU and memory requests/limits of containers based on historical usage. Instead of adding more replicas (HPA), it makes existing Pods bigger or smaller.

HPA	VPA
Scales horizontally (more replicas)	Scales vertically (bigger containers)
Good for stateless, traffic-based workloads	Good for stateful workloads, batch jobs
No downtime during scaling	May require Pod restart for changes
Needs replicas: multiple for effect	Works with single-replica workloads

❏ HPA + VPA Conflict Warning

HPA and VPA should not both target CPU/memory of the same workload simultaneously. HPA scales out while VPA scales up — they conflict. Use HPA for traffic-driven apps, VPA for resource-right-sizing. Use VPA in 'Off' or 'Initial' mode with HPA for safe coexistence.

Chapter 15: API Groups, CRDs, Helm & Kustomize

15.1 Kubernetes API Groups

Kubernetes API is organized into groups. The core group (legacy) contains fundamental resources (Pods, Services, Nodes). Named groups are for newer resources and extensions.

API Group	API Version	Example Resources
Core (legacy)	v1	Pod, Service, ConfigMap, Secret, Node, PV, PVC
apps	apps/v1	Deployment, ReplicaSet, DaemonSet, StatefulSet
batch	batch/v1	Job, CronJob
autoscaling	autoscaling/v2	HorizontalPodAutoscaler
networking.k8s.io	networking.k8s.io/v1	Ingress, NetworkPolicy
rbac.authorization.k8s.io	rbac.authorization.k8s.io/v1	Role, ClusterRole, RoleBinding
storage.k8s.io	storage.k8s.io/v1	StorageClass

15.2 Custom Resource Definitions (CRD)

CRDs extend the Kubernetes API by allowing users to define their own resource types. Once a CRD is created, you can use `kubectl` to create, get, list, delete, and watch instances of your custom resource — just like built-in resources.

□ Analogy

If Kubernetes is a restaurant with a fixed menu (built-in resources), CRDs let you add your own dishes. Once you add 'Pizza' to the menu (create the CRD), any customer (user) can order 'Pizza' (create custom resources) using the standard waiter (`kubectl`).

```
apiVersion: apiextensions.k8s.io/v1
kind: CustomResourceDefinition
metadata:
  name: databases.myorg.io
spec:
  group: myorg.io
  names:
    kind: Database
    plural: databases
    singular: database
  scope: Namespaced
  versions:
  - name: v1
    served: true
    storage: true
    schema:
      openAPIV3Schema:
        type: object
        properties:
          spec:
```

```
type: object
properties:
  size:
    type: string
```

15.3 Helm — The Kubernetes Package Manager

Helm is the de-facto package manager for Kubernetes. It uses Charts — bundles of pre-configured Kubernetes resources — to define, install, and upgrade applications. Helm handles templating (go templates), versioning, and release management.

Core Helm Concepts

Concept	Description
Chart	A Helm package containing templates + default values
Release	An installed instance of a Chart in a cluster
Repository	A collection of Charts (like artifact hub, bitnami)
values.yaml	Default configuration values for a Chart
templates/	Kubernetes manifest templates with Go template syntax

```
# Common Helm commands
helm repo add bitnami https://charts.bitnami.com/bitnami
helm repo update
helm search repo bitnami/nginx
helm install my-nginx bitnami/nginx
helm install my-nginx bitnami/nginx -f custom-values.yaml
helm upgrade my-nginx bitnami/nginx --set replicas=3
helm rollback my-nginx 1
helm uninstall my-nginx
helm list
helm status my-nginx
helm template my-nginx bitnami/nginx # Render templates locally
```

15.4 Kustomize — Configuration Customization

Kustomize provides a template-free way to customize Kubernetes configurations. Instead of templating, it uses overlays and patches to transform base configurations without modifying the originals. It is built directly into kubectl (kubectl apply -k).

Kustomize Structure

```
base/                                # Original configs
  deployment.yaml
  service.yaml
  kustomization.yaml                # base kustomization

overlays/
```

```
dev/
  kustomization.yaml # dev-specific patches
  patch-replicas.yaml
production/
  kustomization.yaml # production-specific patches
  patch-resources.yaml
```

```
# base/kustomization.yaml
resources:
- deployment.yaml
- service.yaml

# overlays/production/kustomization.yaml
bases:
- ../../base
patchesStrategicMerge:
- patch-resources.yaml
images:
- name: my-app
  newTag: v2.1.0

# Apply with kubectl
kubectl apply -k overlays/production/
```

Helm	Kustomize
Template-based (Go templates)	Patch/overlay-based (no templating)
Packages and distributes charts	Customizes existing manifests
Release management & rollback	No built-in release tracking
Better for distributing apps	Better for environment customization
External tool (helm CLI)	Built into kubectl

Chapter 16: Deployment Strategies

Choosing the right deployment strategy determines how your application transitions between versions. Each strategy involves trade-offs between risk, downtime, resource usage, and complexity.

16.1 Rolling Update

The default Kubernetes strategy. Gradually replaces old Pods with new ones. At any point, a mix of old and new version Pods may serve traffic. Controlled via maxSurge and maxUnavailable parameters.

- Zero downtime when configured correctly
- Works natively with Deployments
- Risk: mixed versions serve traffic simultaneously (ensure backward compatibility)

16.2 Recreate

All old Pods are killed before any new Pods are started. Simple but causes downtime equal to the time it takes to start new Pods.

- Guaranteed no mixed-version traffic
- Causes downtime — unsuitable for production services requiring high availability
- Use for: strict version incompatibility, dev/test environments

16.3 Blue-Green Deployment

Two identical production environments (Blue = current, Green = new) run simultaneously. Traffic is switched all at once from Blue to Green using a Service selector change or load balancer rule update. Blue remains as instant rollback.

□ DIAGRAM: Blue-Green Deployment

Two parallel stacks: 'Blue Stack' (v1, 3 Pods) and 'Green Stack' (v2, 3 Pods). A 'Service / Load Balancer' box at top with an arrow that currently points to Blue Stack. After switch: arrow moves to Green Stack instantly. Blue Stack remains idle for rollback. Resources needed: 2x normal capacity.

[Insert diagram here — instructions in Appendix C]

- Instant rollback — just switch selector back to Blue
- No mixed-version traffic during switch
- Requires 2x infrastructure cost during transition

16.4 Canary Deployment

A small percentage of traffic (5-10%) is routed to the new version while the majority continues on the old version. If the canary performs well, traffic is gradually shifted to 100%. If issues appear, roll back only the canary.

□ DIAGRAM: Canary Deployment

A Service routing traffic with weighted distribution: 90% to 'Stable Deployment (v1, 9 Pods)' and 10% to 'Canary Deployment (v2, 1 Pod)'. Monitoring dashboard on the side showing error rates, latency.
After canary success: canary becomes the new stable at 100%.
[Insert diagram here — instructions in Appendix C]

- Minimal blast radius — only small percentage of users affected by new version issues
- Real production traffic validation before full rollout
- Requires traffic splitting (Ingress weights, service mesh)

Strategy	Downtime	Rollback Speed	Resource Cost	Mixed Versions
Rolling Update	None	Medium (re-roll)	Low	Yes
Recreate	Yes	Fast	Low	No
Blue-Green	None	Instant	2x	No
Canary	None	Fast	Slight extra	Yes (controlled)

Chapter 17: Probes — Liveness, Readiness & Startup

Kubernetes probes allow the kubelet to check the health of containers at runtime. They determine whether a container should receive traffic, be restarted, or be given more time to start up.

□ Analogy

Probes are like a hospital monitoring system: Liveness probe checks 'Is the patient alive?' (restart if not). Readiness probe checks 'Is the patient ready to see visitors?' (remove from traffic if not). Startup probe checks 'Is the patient done waking up from surgery?' (don't check too early).

17.1 Liveness Probe

Determines if a container is running correctly. If the liveness probe fails, kubelet kills the container and restarts it according to the restartPolicy. Use for detecting deadlocks or frozen states that don't crash the container.

17.2 Readiness Probe

Determines if a container is ready to accept traffic. If the readiness probe fails, the Pod's IP is removed from the Service Endpoints — it stops receiving traffic but is NOT restarted. Use for graceful startup and temporary unavailability.

17.3 Startup Probe

Provides slow-starting containers extra time to initialize before liveness/readiness probes kick in. While the startup probe is running, liveness and readiness probes are disabled. Once the startup probe succeeds, liveness takes over.

□ DIAGRAM: Probe Timeline

Horizontal timeline for a container lifecycle: (1) Container starts → (2) Startup probe runs (liveness disabled) → (3) Startup probe succeeds → (4) Liveness + Readiness probes begin → (5) Readiness passes → Pod added to Service Endpoints → (6) If Liveness fails → Container restarted. If Readiness fails → removed from Endpoints but not restarted.

[Insert diagram here — instructions in Appendix C]

17.4 Probe Configuration

```
livenessProbe:
  httpGet:
    path: /healthz
    port: 8080
  initialDelaySeconds: 30    # Wait before first probe
  periodSeconds: 10         # Probe every 10 seconds
  failureThreshold: 3       # Fail 3 times before action
  successThreshold: 1       # Successes needed to pass
  timeoutSeconds: 5         # Probe timeout
```

```
readinessProbe:
  httpGet:
    path: /ready
    port: 8080
  initialDelaySeconds: 5
  periodSeconds: 5

startupProbe:
  httpGet:
    path: /healthz
    port: 8080
  failureThreshold: 30      # 30 * 10s = 5 minutes startup time
  periodSeconds: 10
```

Probe Mechanisms

Mechanism	How It Works	Best For
httpGet	HTTP GET to specified path/port; 2xx-3xx = success	HTTP/HTTPS services
tcpSocket	TCP connection to port; connect = success	Non-HTTP services, databases
exec	Runs command in container; exit code 0 = success	Custom health logic
grpc	gRPC health check protocol	gRPC services

Appendix A: Diagram Instructions

This appendix provides detailed visual descriptions for all diagrams referenced throughout the textbook. Use these instructions to recreate diagrams in tools such as draw.io, Lucidchart, Figma, or Miro.

A.1 General Diagram Style Guide

- Background: White (#FFFFFF) or light gray (#F8F8F8)
- Node boxes: Navy border (#1B2A4A), white fill, Arial 12pt labels
- Pod boxes: Blue fill (#D6E4F7), navy border, rounded corners
- Arrows: Dark gray (#4A4A4A), 1.5px width, standard arrowheads
- Warning/note boxes: Orange background (#FAE8DF), orange border
- Success/OK elements: Green border (#1E6B3C) or fill (#D4EDDA)
- Failure/X elements: Red (#C0392B) with X marks

A.2 Kubernetes Architecture Overview (Chapter 11)

Draw a two-section diagram separated by a horizontal dashed line labeled 'Cluster Boundary'. The top section (labeled 'Control Plane', gray background) contains: API Server (large central rectangle, blue), etcd (cylinder/database icon, teal), Scheduler (rectangle), Controller Manager (rectangle), connected by bidirectional arrows to API Server. The bottom section (labeled 'Worker Node') contains: kubelet (rectangle), kube-proxy (rectangle), Container Runtime (rectangle), and 3-4 Pod boxes arranged in a grid. Show `kubectl` → API Server arrow from outside the cluster boundary.

A.3 Service Traffic Flow (Chapter 7)

Draw a vertical flow diagram showing: Internet → (arrow labeled HTTPS:443) → Cloud Load Balancer → (arrow) → NodePort (30080) on Worker Node → kube-proxy (iptables rules) → Service (ClusterIP) → Endpoints → Pod. Show the ClusterIP as a virtual floating IP with dashed border. Include DNS query: App → CoreDNS → Service FQDN → ClusterIP.

Appendix B: Complete kubectl Command Reference

B.1 Context & Cluster Management

```
kubectl config get-contexts          # List all contexts
kubectl config use-context <name>    # Switch context
kubectl config current-context        # Show current context
kubectl cluster-info                  # Cluster endpoint info
kubectl get nodes                     # List nodes
kubectl describe node <name>         # Node details
```

B.2 Namespace Commands

```
kubectl get namespaces
kubectl create namespace production
kubectl config set-context --current --namespace=production
kubectl get all -n production
```

B.3 Workload Commands

```
kubectl get pods -A                  # All namespaces
kubectl get pods -o wide              # With node info
kubectl get pods -l app=nginx         # Label filter
kubectl get pods --watch              # Watch changes
kubectl describe pod <name>
kubectl logs <pod> --previous         # Previous container logs
kubectl logs <pod> --since=1h        # Last hour logs
kubectl exec -it <pod> -- sh
kubectl port-forward pod/<name> 8080:80 # Port forward
kubectl cp <pod>:/path/file ./local/file # Copy from Pod
```

B.4 Debugging Commands

```
kubectl get events --sort-by='.lastTimestamp'
kubectl get events -n <namespace>
kubectl top pods --sort-by=cpu
kubectl top nodes
kubectl run debug --image=busybox -it --rm -- sh # Temp debug pod
kubectl debug node/<node-name> -it --image=ubuntu # Node debug
```

Appendix C: Glossary of Kubernetes Terms

Term	Definition
Admission Controller	Plugin that intercepts API requests to validate or mutate objects before persistence
CNI	Container Network Interface — plugin standard for configuring Pod networking
CRD	Custom Resource Definition — extends Kubernetes API with custom resource types
CRI	Container Runtime Interface — standard for container runtimes to integrate with kubelet
ETCD	Distributed key-value store serving as Kubernetes' backing store for all cluster data
Helm	Package manager for Kubernetes, using Charts to define and deploy applications
HPA	Horizontal Pod Autoscaler — scales Pod replicas based on metrics
Ingress	API object defining HTTP/HTTPS routing rules from external traffic to Services
kubelet	Node agent that ensures containers described in PodSpecs are running
kube-proxy	Node component that maintains network rules for Service traffic routing
Namespace	Virtual cluster within a Kubernetes cluster for resource isolation
Node	A worker machine (VM or physical) in the Kubernetes cluster
OIDC	OpenID Connect — authentication protocol used for Kubernetes identity federation
Pod	Smallest deployable unit containing one or more containers sharing network/storage
RBAC	Role-Based Access Control — authorization mechanism based on roles
ReplicaSet	Controller ensuring a specified number of Pod replicas are running
Service	Stable network endpoint (virtual IP + DNS) routing traffic to Pods
StatefulSet	Controller for stateful applications providing stable identity and persistent storage
VPA	Vertical Pod Autoscaler — adjusts container resource requests/limits automatically
Volume	Directory accessible to containers in a Pod, with defined lifecycle and storage backend

tech__.talks